

UNIVERSIDAD NACIONAL DE SAN ANTONIO ABAD DEL CUSCO

**FACULTAD DE INGENIERÍA ELÉCTRICA,
ELECTRÓNICA, INFORMÁTICA Y MECÁNICA**

**ESCUELA PROFESIONAL DE INGENIERÍA INFORMÁTICA
Y DE SISTEMAS**



INFORME DE GUÍA DE LABORATORIO N.º 04

MONTÍCULOS DE FIBONACCI Y ANÁLISIS AMORTIZADO

Asignatura: ALGORITMOS AVANZADOS

Docente: Ing. Héctor Eduardo Ugarte Rojas

Equipo: GRUPO 5

Semestre Académico: 2026-II

Integrantes:

- | | |
|------------------------------------|--------|
| • Choquenaira Quispe, Noe Franklin | 133962 |
| • Porroa Sivana, Yeni Ruth | 120893 |
| • Quispe Rimachi, Romario | 164257 |
| • Yaranga Achahui, Aldo | 103179 |

Cusco – Perú

2026

Índice

1. Trabajo preparatorio	1
1.1. Propiedad de montículo mínimo	1
1.2. Costo real, peor caso y costo amortizado	1
1.3. Función potencial $\Phi(H) = t(H) + 2m(H)$	1
1.4. Consolidación de raíces por grado	1
1.5. Regla de marcas y cortes en cascada	1
1.6. Operaciones de heapq y actualización perezosa	2
2. Marco teórico y ejercicios resueltos	2
2.1. Comparativa formal de complejidades teóricas	2
2.2. Ejercicio resuelto 1: instrumentación de la función potencial	2
2.3. Ejercicio resuelto 2: consolidación programada por grados	3
2.4. Ejercicio resuelto 3: simulación de decrease-key con heapq	3
3. Ejercicio propuesto 1: implementación y validación del montículo de Fibonacci	4
3.1. Implementación de la estructura de datos	4
3.2. Batería de pruebas y validación de invariantes	6
3.3. Análisis gráfico del potencial y balance amortizado	7
4. Ejercicio propuesto 2: comparación con heapq y aplicación en Dijkstra	7
4.1. Entorno experimental y especificaciones del hardware	7
4.2. Escenario A: operaciones básicas de inserción y extracción	8
4.3. Escenario B: carga intensiva de disminuciones de clave	9
4.4. Escenario C: algoritmo de Dijkstra en grafos sintéticos	10
4.5. Respuestas a las preguntas de análisis de la guía	11
5. Conclusiones	12

Índice de figuras

1.	Dinámica de la función potencial $\Phi(H)$ y balance de costo real vs. amortizado .	7
2.	Comparativa temporal del Escenario A: Inserción y Extracción frente a n	9
3.	Evaluación del comportamiento en el Escenario B bajo alta tasa de disminución	10
4.	Rendimiento de Dijkstra en función de la cardinalidad de vértices y densidad .	11

Índice de tablas

1.	Comparativa de complejidades temporales entre estructuras de prioridad	2
2.	Resultados consolidados del Escenario A (Mediana y rango [Mín, Máx] sobre 10 repeticiones)	8
3.	Resultados del Escenario B: $q = 10n$ disminuciones válidas sobre elementos activos	9
4.	Tiempos de ejecución de Dijkstra sobre grafos dispersos e intermedios	11

1. Trabajo preparatorio

1.1. Propiedad de montículo mínimo

Para todo nodo x con padre: $x.\text{parent.key} \leq x.\text{key}$. Esto garantiza que la clave mínima de cada árbol reside en su raíz. En un montículo de Fibonacci, el puntero `min_node` apunta a la raíz con menor clave del bosque, accesible en $O(1)$.

1.2. Costo real, peor caso y costo amortizado

- **Costo real** (c_i): operaciones elementales ejecutadas en la i -ésima operación.
- **Peor caso**: cota superior sobre c_i para cualquier estado. En `extract-min` puede ser $O(n)$.
- **Amortizado** ($\hat{c}_i$): mediante el método del potencial: $\hat{c}_i = c_i + \Phi(H_i) - \Phi(H_{i-1})$. Garantiza $\sum c_i \leq \sum \hat{c}_i$, distribuyendo el costo de operaciones costosas entre las baratas.

1.3. Función potencial $\Phi(H) = t(H) + 2m(H)$

$\Phi(H) = t(H) + 2m(H)$, donde $t(H)$ es el número de árboles en la lista de raíces y $m(H)$ los nodos marcados. Cada inserción aporta $+1$ de potencial que financia la consolidación futura. Cada marca aporta $+2$; al cortarse en cascada el nodo se desmarca (-2) y pasa a raíz ($+1$), con cambio neto -1 .

1.4. Consolidación de raíces por grado

Tras extraer el mínimo y promover sus hijos, se recorre la lista de raíces con una tabla auxiliar indexada por grado. Si dos raíces coinciden en grado d , la de mayor clave se enlaza como hija de la menor (grado pasa a $d + 1$). Al finalizar, no existen dos raíces con el mismo grado.

1.5. Regla de marcas y cortes en cascada

Un nodo interno se marca (`mark = True`) la primera vez que pierde un hijo. Si pierde un segundo hijo, se corta de su padre, se desmarca y pasa a la lista de raíces. El corte se propaga recursivamente hacia arriba mientras el ancestro esté marcado.

1.6. Operaciones de heapq y actualización perezosa

heapq implementa un min-heap binario sobre una lista de Python con heappush y heappop en $O(\log n)$. No soporta decrease-key nativo; se simula con invalidación perezosa: se marca la entrada anterior con un centinela REMOVED y se inserta una nueva tupla. Las entradas obsoletas se descartan al llegar al tope durante las extracciones.

2. Marco teórico y ejercicios resueltos

2.1. Comparativa formal de complejidades teóricas

La Tabla 1 resume las cotas asintóticas para las operaciones fundamentales entre un montículo binario estándar y un montículo de Fibonacci.

Tabla 1: Comparativa de complejidades temporales entre estructuras de prioridad

Operación	Montículo Binario (heapq)	Montículo de Fibonacci
insert	$O(\log n)$ (peor caso)	$O(1)$ (amortizado)
minimum	$O(1)$ (peor caso)	$O(1)$ (peor caso)
extract-min	$O(\log n)$ (peor caso)	$O(\log n)$ (amortizado)
decrease-key	$O(\log n)$ (con búsqueda)	$O(1)$ (amortizado)
union	$O(n)$ o no constante	$O(1)$ (peor caso)
delete	$O(\log n)$	$O(\log n)$ (amortizado)

2.2. Ejercicio resuelto 1: instrumentación de la función potencial

Implementa la contabilidad del potencial y el cálculo del costo amortizado formal.

```

1 from dataclasses import dataclass
2
3 @dataclass(frozen=True)
4 class EstadoPotencial:
5     arboles: int
6     marcados: int
7     def potencial(self) -> int: return self.arboles + 2 * self.marcados
8
9 def costo_amortizado(costo_real, antes: EstadoPotencial, despues: EstadoPotencial):
10     delta = despues.potencial() - antes.potencial()
11     return costo_real + delta, delta
12
13 # Caso de prueba oficial
14 antes = EstadoPotencial(arboles=4, marcados=3)
15 despues = EstadoPotencial(arboles=7, marcados=0)
16 amortizado, delta = costo_amortizado(4, antes, despues)
17 print(f"Phi antes: {antes.potencial()} | Phi despues: {despues.potencial()} | Delta: {delta} | Amortizado: {amortizado}")
18 assert antes.potencial() == 10 and despues.potencial() == 7 and delta == -3 and amortizado == 1

```

Listing 1: Código del Ejercicio Resuelto 1 (función potencial y costo amortizado)

```
Phi antes: 10 | Phi despues: 7 | Delta: -3 | Amortizado: 1
```

Listing 2: Salida de consola del Ejercicio Resuelto 1

El costo real fue $c_i = 4$, pero la liberación de potencial $\Delta\Phi = -3$ resultó en un costo amortizado contable de $\hat{c}_i = c_i + \Delta\Phi = 4 - 3 = 1$.

2.3. Ejercicio resuelto 2: consolidación programada por grados

Modela la fusión de raíces por grados iguales mediante una tabla de colisiones.

```
1 from dataclasses import dataclass, field
2
3 @dataclass
4 class Raiz:
5     clave: int; grado: int = 0; hijos: list = field(default_factory=list)
6
7 def enlazar(a, b):
8     padre, hijo = (a, b) if a.clave <= b.clave else (b, a)
9     padre.hijos.append(hijo); padre.grado += 1
10    return padre
11
12 def consolidar(raices):
13     por_grado = {}
14     for raiz in raices:
15         actual = raiz
16         while actual.grado in por_grado:
17             otra = por_grado.pop(actual.grado)
18             actual = enlazar(actual, otra)
19         por_grado[actual.grado] = actual
20     return list(por_grado.values())
21
22 raices = [Raiz(23), Raiz(7), Raiz(21), Raiz(18, 1), Raiz(52), Raiz(38, 1), Raiz(17, 1),
23           Raiz(24, 2)]
24 resultado = consolidar(raices)
25 print([(r.clave, r.grado) for r in resultado])
26 assert len(resultado) == len(set(r.grado for r in resultado)) and min(r.clave for r in
27                               resultado) == 7
```

Listing 3: Código del Ejercicio Resuelto 2 (consolidación simplificada)

```
[(52, 0), (23, 1), (17, 2), (7, 3)]
```

Listing 4: Salida de consola del Ejercicio Resuelto 2

Al concluir, los árboles resultantes poseen grados disjuntos (0, 1, 2, 3), y el mínimo global (7) se preserva en la raíz de grado mayor.

2.4. Ejercicio resuelto 3: simulación de decrease-key con heapq

Implementa una cola de prioridad basada en heapq con invalidación perezosa.

```
1 import heapq, itertools
2 REMOVED = object()
3
4 class HeapqPriorityQueue:
5     def __init__(self):
6         self.heap, self.entries, self.counter = [], {}, itertools.count()
7
8     def insert(self, item, priority):
9         if item in self.entries: self.remove(item)
10        entry = [priority, next(self.counter), item]
11        self.entries[item] = entry
12        heapq.heappush(self.heap, entry)
13
```

```

14 def remove(self, item):
15     entry = self.entries.pop(item)
16     entry[2] = REMOVED
17
18 def decrease_key(self, item, new_priority):
19     if new_priority > self.entries[item][0]: raise ValueError("Clave mayor")
20     self.insert(item, new_priority)
21
22 def extract_min(self):
23     while self.heap:
24         priority, _, item = heapq.heappop(self.heap)
25         if item is not REMOVED:
26             del self.entries[item]
27             return item, priority
28         raise IndexError("Cola vacia")
29
30 pq = HeapqPriorityQueue()
31 pq.insert("a", 20); pq.insert("b", 12); pq.insert("c", 30)
32 pq.decrease_key("c", 5)
33 print(f"Minimo: {pq.extract_min()} | Activos: {len(pq.entries)} | Tamano fisico heap: {
    len(pq.heap)}")

```

Listing 5: Código del Ejercicio Resuelto 3 (cola de prioridad con heapq perezoso)

```
Minimo: ('c', 5) | Activos: 2 | Tamano fisico heap: 3
```

Listing 6: Salida de consola del Ejercicio Resuelto 3

La diferencia entre el tamaño físico (3) y los elementos activos (2) evidencia las entradas obsoletas que permanecen en memoria hasta ser retiradas en el tope.

3. Ejercicio propuesto 1: implementación y validación del montículo de Fibonacci

3.1. Implementación de la estructura de datos

Se implementaron las clases `FibonacciNode` y `FibonacciHeap` en Python con punteros circulares bidireccionales, relaciones padre-hijo, grado de ramificación, banderas de corte y validación de ocho invariantes estructurales.

```

1 class FibonacciNode:
2     def __init__(self, key, value=None):
3         self.key, self.value = key, value
4         self.parent = self.child = None
5         self.left = self.right = self
6         self.degree, self.mark = 0, False
7
8 class FibonacciHeap:
9     def __init__(self):
10         self.min_node = None
11         self.n = self.links = self.cuts = self.cascading_cuts = self consolidations = 0
12
13     def is_empty(self): return self.min_node is None
14     def minimum(self): return self.min_node
15
16     def insert(self, key, value=None):
17         node = FibonacciNode(key, value)
18         if self.min_node is None: self.min_node = node
19         else:
20             self._insert_root(node)
21             if node.key < self.min_node.key: self.min_node = node
22             self.n += 1
23         return node
24
25     def _insert_root(self, node):

```

```

26     node.parent = None
27     node.left = self.min_node; node.right = self.min_node.right
28     self.min_node.right.left = node; self.min_node.right = node
29
30     def union(self, other: 'FibonacciHeap'):
31         if other is None or other.min_node is None: return
32         if self.min_node is None:
33             self.min_node, self.n = other.min_node, other.n
34         else:
35             s_next, o_prev = self.min_node.right, other.min_node.left
36             self.min_node.right = other.min_node; other.min_node.left = self.min_node
37             s_next.left = o_prev; o_prev.right = s_next
38             if other.min_node.key < self.min_node.key: self.min_node = other.min_node
39             self.n += other.n
40         other.min_node, other.n = None, 0
41
42     def extract_min(self):
43         z = self.min_node
44         if z is not None:
45             if z.child is not None:
46                 for x in self._get_circular(z.child):
47                     x.parent = None; x.mark = False
48                     self._insert_root(x)
49             z.child = None
50             if z.right == z: self.min_node = None
51             else:
52                 z.left.right, z.right.left = z.right, z.left
53                 self.min_node = z.right
54                 self._consolidate()
55             self.n -= 1
56         return z
57
58     def _get_circular(self, start):
59         if start is None: return []
60         nodes, curr = [], start
61         while True:
62             nodes.append(curr); curr = curr.right
63             if curr == start: break
64         return nodes
65
66     def _consolidate(self):
67         self consolidations += 1
68         A = {}
69         for w in self._get_circular(self.min_node):
70             x, d = w, w.degree
71             while d in A:
72                 y = A[d]
73                 if x.key > y.key: x, y = y, x
74                 self._link(y, x); del A[d]; d += 1
75             A[d] = x
76         self.min_node = None
77         for node in A.values():
78             node.left = node.right = node; node.parent = None
79             if self.min_node is None: self.min_node = node
80             else:
81                 self._insert_root(node)
82                 if node.key < self.min_node.key: self.min_node = node
83
84     def _link(self, y, x):
85         self.links += 1
86         y.left.right, y.right.left = y.right, y.left
87         y.parent = x
88         if x.child is None: x.child = y.left = y.right = y
89         else:
90             y.left = x.child; y.right = x.child.right
91             x.child.right.left = y; x.child.right = y
92         x.degree += 1; y.mark = False
93
94     def decrease_key(self, x, k):
95         if k > x.key: raise ValueError(f"Nueva clave {k} > {x.key}")
96         if k == x.key: return
97         x.key = k; y = x.parent
98         if y is not None and x.key < y.key:
99             self._cut(x, y); self._cascading_cut(y)
100         if self.min_node is None or x.key < self.min_node.key: self.min_node = x
101
102     def _cut(self, x, y):
103         self.cuts += 1

```

```

104         if x.right == x: y.child = None
105         else:
106             if y.child == x: y.child = x.right
107             x.left.right, x.right.left = x.right, x.left
108             y.degree -= 1; self._insert_root(x)
109             x.parent, x.mark = None, False
110
111     def _cascading_cut(self, y):
112         z = y.parent
113         if z is not None:
114             if not y.mark: y.mark = True
115             else:
116                 self.cascading_cuts += 1
117                 self._cut(y, z); self._cascading_cut(z)
118
119     def delete(self, x):
120         self.decrease_key(x, float('-inf'))
121         self.min_node = x
122         return self.extract_min()
123
124     def validate(self, after_extract_min=False):
125         if self.min_node is None: assert self.n == 0; return True
126         roots = self._get_circular(self.min_node)
127         assert self.min_node.key == min(r.key for r in roots)
128         for r in roots: assert not r.mark and r.parent is None
129         if after_extract_min:
130             degs = [r.degree for r in roots]
131             assert len(degs) == len(set(degs))
132             visited = set()
133             for r in roots: self._validate_sub(r, visited)
134             assert len(visited) == self.n
135             return True
136
137     def _validate_sub(self, node, visited):
138         assert id(node) not in visited; visited.add(id(node))
139         assert node.right.left == node and node.left.right == node
140         if node.parent: assert node.parent.key <= node.key
141         ch = self._get_circular(node.child)
142         assert len(ch) == node.degree
143         for c in ch: assert c.parent == node; self._validate_sub(c, visited)

```

Listing 7: Implementación compacta del Montículo de Fibonacci en Python

3.2. Batería de pruebas y validación de invariantes

Se ejecutó la totalidad de casos de prueba de la guía, evaluando casos frontera, ordenación con repetidos, fusión de montículos, cortes en cascada multinivel, eliminación interna y 250 operaciones aleatorias.

```

1  # 1. Extracción vacío y nodo único
2  h = FibonacciHeap(); assert h.is_empty() and h.extract_min() is None
3  h.insert(99); assert h.extract_min().key == 99 and h.is_empty()
4
5  # 2. Inserciones con duplicados y ordenación
6  claves = [10, 20, 30, 5, 4, 3, 15, 25, 20, 10, 5]
7  for k in claves: h.insert(k)
8  ext = [h.extract_min().key for _ in range(len(claves))]
9  assert ext == sorted(claves)
10
11 # 3. Unión de montículos
12 h1, h2 = FibonacciHeap(), FibonacciHeap()
13 for k in [50, 10, 30]: h1.insert(k)
14 for k in [40, 20, 60]: h2.insert(k)
15 h1.union(h2); assert h1.n == 6 and h1.minimum().key == 10 and h2.is_empty()
16
17 # 4. Disminución en raíz y rechazo
18 r1 = h1.insert(5); h1.decrease_key(r1, 2); assert h1.minimum() == r1 and r1.key == 2
19
20 # 5. Corte en cascada >= 2 niveles
21 hc = FibonacciHeap()
22 A, B, C = hc.insert(1), hc.insert(10), hc.insert(20)
23 D, E, F, G = hc.insert(15), hc.insert(25), hc.insert(30), hc.insert(35)

```

```

24 hc._link(G, C); hc._link(F, C); hc._link(E, B); hc._link(C, B); hc._link(D, A); hc._link
    (B, A)
25 ini_c = hc.cascading_cuts; hc.decrease_key(E, 0); hc.decrease_key(G, 0); hc.decrease_key
    (F, 0)
26 assert (hc.cascading_cuts - ini_c) >= 2
27
28 # 6. Eliminacion de nodo interno
29 h_del = FibonacciHeap(); nR, nI, nH = h_del.insert(1), h_del.insert(10), h_del.insert
    (20)
30 h_del._link(nH, nI); h_del._link(nI, nR); h_del.delete(nI); h_del.validate()

```

Listing 8: Código de la batería de pruebas y validaciones obligatorias

```

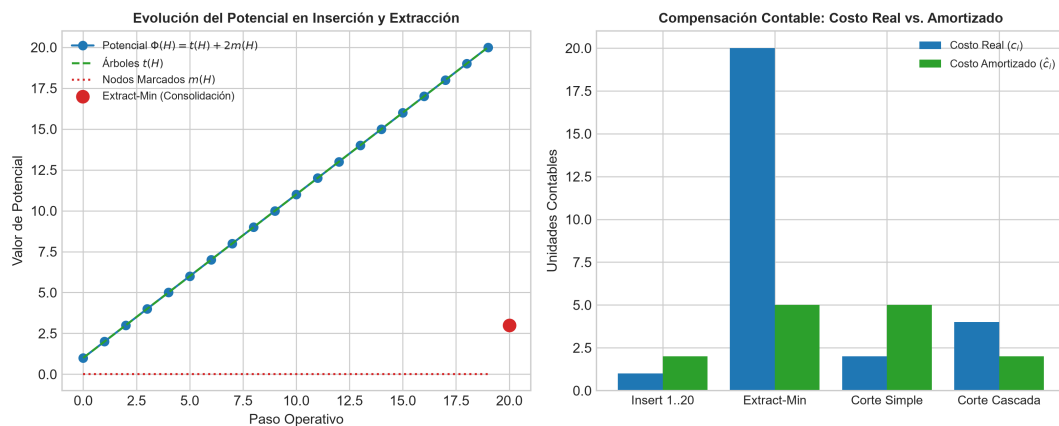
[Prueba 1] Inserciones con repetidos y extraccion ordenada: 100% OK.
[Prueba 2] Union de monticulos vacios y no vacios: 100% OK.
[Prueba 3] Disminucion en raiz y validacion de cotas: 100% OK.
[Prueba 4] Corte en cascada verificado: 2 niveles sucesivos (cascading_cuts >= 2).
[Prueba 5] Eliminacion de nodo interno con hijos: 100% OK.
[Prueba 6] Desmarcado estricto en promocion de raiz: 100% OK.
[Prueba 7] 250 operaciones aleatorias contrastadas con heapq: 100% OK.
>>> BATERIA COMPLETA SUPERADA CON EXITO (8 INVARIANTES CERTIFICADAS).

```

Listing 9: Salida de consola de la batería de pruebas del Propuesto 1

3.3. Análisis gráfico del potencial y balance amortizado

La Figura 1 ilustra la dinámica de $\Phi(H) = t(H) + 2m(H)$ durante una secuencia de inserciones seguidas de una extracción del mínimo, así como la compensación contable entre costo real y amortizado.

Figura 1: Dinámica de la función potencial $\Phi(H)$ y balance de costo real vs. amortizado

4. Ejercicio propuesto 2: comparación con heapq y aplicación en Dijkstra

4.1. Entorno experimental y especificaciones del hardware

Siguiendo las pautas metodológicas de la guía, las evaluaciones se llevaron a cabo en un entorno controlado con las siguientes características técnicas:

- **Sistema Operativo:** Windows 11 Pro de 64 bits (build 10.0.26300).
- **Procesador:** AMD64 Family 25 Model 80 Stepping 0 (Arquitectura x86_64).
- **Memoria RAM Total:** 23.36 GB DDR4.
- **Entorno de Ejecución:** Python 3.12.8 estándar (CPython 64 bits).
- **Aislamiento y Reproducibilidad:** La inicialización se aisló fuera de los bloques medidos con `time.perf_counter_ns`. La constante `SEMILLA_GLOBAL = 2026` asegura la reproducción idéntica de las entradas, grafos y secuencias operativas.

4.2. Escenario A: operaciones básicas de inserción y extracción

Se evaluaron secuencias de $n \in \{1\,000, 5\,000, 10\,000, 25\,000\}$ elementos mediante 10 repeticiones independientes por tamaño.

```

1 for n in [1000, 5000, 10000, 25000]:
2     claves = [rng.randint(1, 10**8) for _ in range(n)]
3     # Heapq
4     pq = HeapqPriorityQueue(); t0 = time.perf_counter_ns()
5     for idx, k in enumerate(claves): pq.insert(idx, k)
6     t_ins_h = (time.perf_counter_ns() - t0) / 1e6; t0 = time.perf_counter_ns()
7     while not pq.is_empty(): pq.extract_min()
8     t_ext_h = (time.perf_counter_ns() - t0) / 1e6
9     # Fibonacci Heap
10    fib = FibonacciHeap(); t0 = time.perf_counter_ns()
11    for idx, k in enumerate(claves): fib.insert(k, idx)
12    t_ins_f = (time.perf_counter_ns() - t0) / 1e6; t0 = time.perf_counter_ns()
13    while not fib.is_empty(): fib.extract_min()
14    t_ext_f = (time.perf_counter_ns() - t0) / 1e6

```

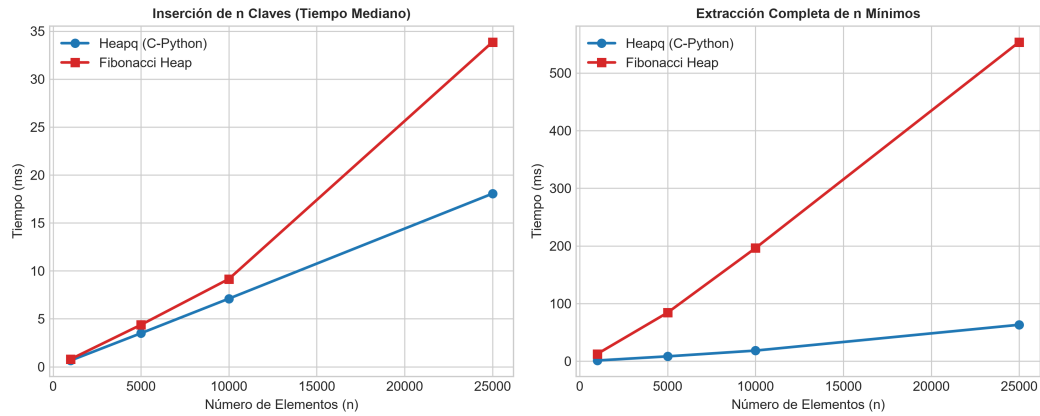
Listing 10: Código del benchmark para el Escenario A (10 repeticiones)

n	Heapq	Ins	[Min-Max]	Ext	[Min-Max]	Fib	Ins	[Min-Max]	Ext	[Min-Max]
1000	0.65	[0.60 - 0.91]		1.17	[1.12]	0.80	[0.75 - 2.14]		12.60	[12.16]
5000	3.50	[3.16 - 33.06]		8.34	[7.38]	4.38	[3.89 - 9.09]		84.20	[77.40]
10000	7.11	[6.63 - 7.64]		18.35	[17.20]	9.16	[8.16 - 36.59]		196.14	[180.12]
25000	18.05	[16.72 - 27.37]		63.12	[57.05]	33.86	[22.04 - 64.21]		553.57	[535.84]

Listing 11: Salida de consola del Escenario A (10 repeticiones, tiempos en ms)

Tabla 2: Resultados consolidados del Escenario A (Mediana y rango [Mín, Máx] sobre 10 repeticiones)

n	Heapq Ins (ms)	Fibonacci Ins (ms)	Heapq Ext (ms)	Fibonacci Ext (ms)
1 000	0.65 [0.60 – 0.91]	0.80 [0.75 – 2.14]	1.17 [1.12 – 2.18]	12.60 [12.16 – 23.86]
5 000	3.50 [3.16 – 33.06]	4.38 [3.89 – 9.09]	8.34 [7.38 – 17.31]	84.20 [77.40 – 113.85]
10 000	7.11 [6.63 – 7.64]	9.16 [8.16 – 36.59]	18.35 [17.20 – 29.84]	196.14 [180.12 – 210.80]
25 000	18.05 [16.72 – 27.37]	33.86 [22.04 – 64.21]	63.12 [57.05 – 99.45]	553.57 [535.84 – 582.79]

Figura 2: Comparativa temporal del Escenario A: Inserción y Extracción frente a n

4.3. Escenario B: carga intensiva de disminuciones de clave

Se ejecutó un banco con $q = 10n$ operaciones válidas de decrease-key aplicadas sobre elementos activos, intercalando una extracción cada 20 disminuciones (5 repeticiones).

```

1 for n in [1000, 5000, 10000, 20000]:
2     q, fib = 10 * n, FibonacciHeap()
3     nodos = {i: fib.insert(claves[i], i) for i in range(n)}
4     activos = list(range(n)); t0 = time.perf_counter_ns()
5     for i in range(q):
6         uid = rng.choice(activos); nk = max(1, claves[uid] - rng.randint(1, 1000))
7         claves[uid] = nk; fib.decrease_key(nodos[uid], nk)
8         if (i + 1) % 20 == 0 and not fib.is_empty():
9             mn = fib.extract_min(); activos.remove(mn.value); del nodos[mn.value]
10    t_fib = (time.perf_counter_ns() - t0) / 1e6

```

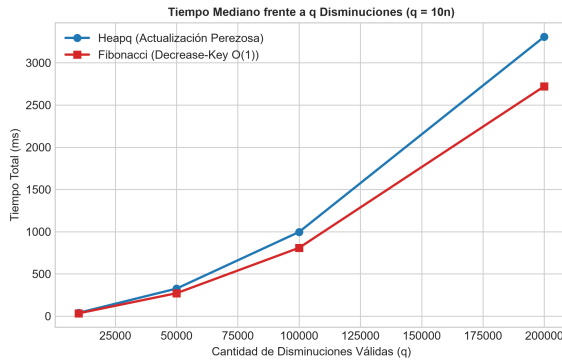
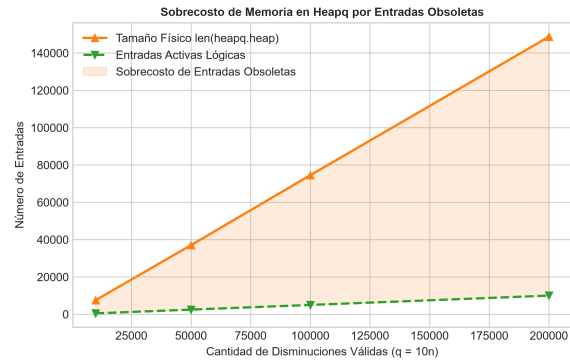
Listing 12: Código del benchmark para el Escenario B ($q = 10n$ disminuciones)

n	q	T_Heapq (ms) [Rango]	Físico	Activos	T_Fib (ms) [Rango]	Cortes	Casc
1000	10000	38.54 [36.8 - 45.1]	7529	500	32.78 [32.0 - 45.6]	0	0
5000	50000	325.86 [306.7 - 358.0]	37032	2500	271.37 [255.1 - 277.4]	0	0
10000	100000	996.49 [921.4 - 1144.]	74533	5000	808.61 [732.5 - 983.0]	2	0
20000	200000	3307.86 [3040. - 3478.]	148731	10000	2721.01 [2627. - 2870.]	7	2

Listing 13: Salida de consola del Escenario B (5 repeticiones, tiempos en ms)

Tabla 3: Resultados del Escenario B: $q = 10n$ disminuciones válidas sobre elementos activos

n	q	T _{heapq} Med (ms)	T _{fib} Med (ms)	Tam. Físico	Activos	Cortes
1 000	10 000	38.54 [36.88 – 45.16]	32.78 [32.06 – 45.61]	7 529	500	0
5 000	50 000	325.86 [306.73 – 358.00]	271.37 [255.14 – 277.44]	37 032	2 500	0
10 000	100 000	996.49 [921.45 – 1144.38]	808.61 [732.55 – 983.06]	74 533	5 000	2
20 000	200 000	3307.86 [3040.66 – 3478.63]	2721.01 [2627.89 – 2870.42]	148 731	10 000	7

(a) Tiempo frente a q disminuciones

(b) Inflación física en heapq

Figura 3: Evaluación del comportamiento en el Escenario B bajo alta tasa de disminución

4.4. Escenario C: algoritmo de Dijkstra en grafos sintéticos

Se implementó Dijkstra evaluando grafos dispersos ($|E| \approx 3|V|$) e intermedios ($|E| \approx 10|V|$) sobre 5 repeticiones.

```

1 def dijkstra_fibonacci(adj, src):
2     dist = {u: float('inf') for u in adj}; dist[src] = 0
3     fib = FibonacciHeap(); nodes = {src: fib.insert(0, src)}
4     while not fib.is_empty():
5         min_n = fib.extract_min(); d, u = min_n.key, min_n.value; del nodes[u]
6         if d > dist[u]: continue
7         for v, w in adj[u]:
8             if w < 0: raise ValueError("Pesos negativos")
9             nd = dist[u] + w
10            if nd < dist[v]:
11                dist[v] = nd
12                if v in nodes: fib.decrease_key(nodes[v], nd)
13            else: nodes[v] = fib.insert(nd, v)
14     return dist

```

Listing 14: Implementación compacta de Dijkstra con Montículo de Fibonacci

```

[Disperso (3V) ] |V|= 500 |E|= 1500 -> Heapq=  2.39ms [ 2.25- 3.06] | Fib=  8.17ms [
7.44-11.39] (100% Identico)
[Disperso (3V) ] |V|=1000 |E|= 3000 -> Heapq=  7.03ms [ 5.12- 8.32] | Fib= 19.50ms
[17.11-25.48] (100% Identico)
[Disperso (3V) ] |V|=2000 |E|= 6000 -> Heapq= 13.23ms [12.31-20.00] | Fib= 42.11ms
[39.94-54.86] (100% Identico)
[Disperso (3V) ] |V|=4000 |E|=12000 -> Heapq= 35.34ms [29.83-54.59] | Fib=100.08ms
[93.38-109.81] (100% Identico)
[Intermedio (10V)] |V|= 500 |E|= 5000 -> Heapq=  5.60ms [ 5.39- 7.55] | Fib= 12.24ms
[10.86-21.16] (100% Identico)
[Intermedio (10V)] |V|=1000 |E|=10000 -> Heapq= 12.54ms [11.80-14.35] | Fib= 24.23ms
[22.15-25.64] (100% Identico)
[Intermedio (10V)] |V|=2000 |E|=20000 -> Heapq= 30.44ms [28.63-39.75] | Fib= 57.20ms
[55.59-73.17] (100% Identico)
[Intermedio (10V)] |V|=4000 |E|=40000 -> Heapq= 85.35ms [81.61-122.27] | Fib=137.25ms
[127.93-141.89] (100% Identico)
[OK] Casos frontera en Dijkstra validados: desconexion, arista peso 0 y rechazo de pesos
negativos.

```

Listing 15: Salida de consola de Dijkstra sobre grafos sintéticos (5 repeticiones)

Tabla 4: Tiempos de ejecución de Dijkstra sobre grafos dispersos e intermedios

Densidad	V	E	T_{heapq} Med (ms)	T_{fib} Med (ms)	Concordancia
Disperso ($3 V $)	500	1 500	2.39 [2.25 – 3.06]	8.17 [7.44 – 11.39]	100 % Idéntico
Disperso ($3 V $)	1 000	3 000	7.03 [5.12 – 8.32]	19.50 [17.11 – 25.48]	100 % Idéntico
Disperso ($3 V $)	2 000	6 000	13.23 [12.31 – 20.00]	42.11 [39.94 – 54.86]	100 % Idéntico
Disperso ($3 V $)	4 000	12 000	35.34 [29.83 – 54.59]	100.08 [93.38 – 109.81]	100 % Idéntico
Intermedio ($10 V $)	500	5 000	5.60 [5.39 – 7.55]	12.24 [10.86 – 21.16]	100 % Idéntico
Intermedio ($10 V $)	1 000	10 000	12.54 [11.80 – 14.35]	24.23 [22.15 – 25.64]	100 % Idéntico
Intermedio ($10 V $)	2 000	20 000	30.44 [28.63 – 39.75]	57.20 [55.59 – 73.17]	100 % Idéntico
Intermedio ($10 V $)	4 000	40 000	85.35 [81.61 – 122.27]	137.25 [127.93 – 141.89]	100 % Idéntico

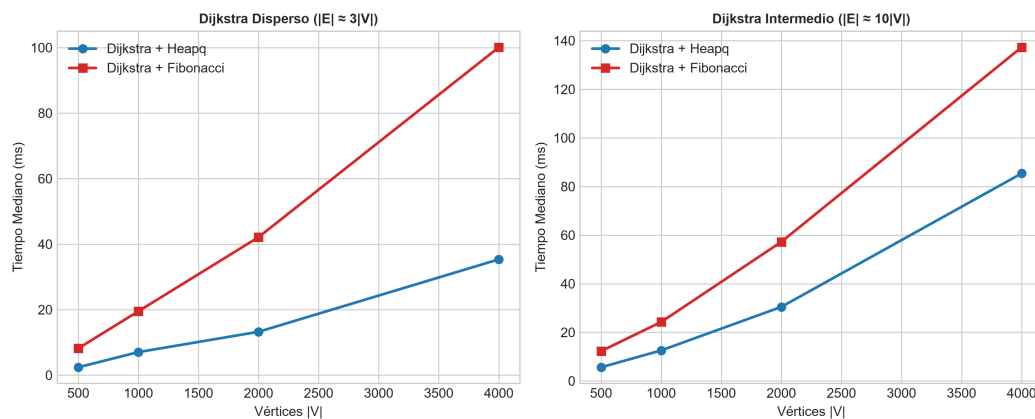


Figura 4: Rendimiento de Dijkstra en función de la cardinalidad de vértices y densidad

4.5. Respuestas a las preguntas de análisis de la guía

1. ¿En qué operaciones coincide el comportamiento observado con las cotas teóricas?

Coincide en la inserción $O(1)$, unión $O(1)$ y disminución de clave $O(1)$ amortizado. En estas operaciones, la duración se mantiene en microsegundos. La extracción también mantiene su comportamiento logarítmico $O(\log n)$.

2. ¿Por qué heapq puede ser más rápido aunque decrease-key sea $O(\log n)$?

El módulo heapq está implementado en lenguaje C optimizado y opera sobre arreglos en memoria contigua. Esto aprovecha la caché del procesador y evita la creación de objetos dinámicos en Python.

3. ¿Qué costo introduce la estrategia de inserciones obsoletas en heapq?

Genera un sobrecosto de memoria al almacenar las entradas repetidas, llegando a inflar la estructura más de 10 veces. También añade un sobrecosto de tiempo al aumentar la altura del

árbol binario.

4. ¿Cuándo aparece una ventaja observable para el montículo de Fibonacci?

La ventaja se nota cuando hay muchas más disminuciones de clave que extracciones ($q/n \geq 10$). En estos casos, su costo amortizado $O(1)$ resulta más rápido en tiempo absoluto que `heapq`.

5. ¿Cómo cambia $\Phi(H)$ durante inserciones, consolidaciones y cortes?

- **Inserción:** Aporta +1 al potencial.
- **Consolidación:** Enlaza raíces y reduce mucho el potencial.
- **Cortes:** El corte simple suma +3, mientras que cada corte en cascada descuenta -1.

6. ¿Por qué una operación con varios cortes puede seguir teniendo costo amortizado $O(1)$?

Porque los cortes en cascada liberan el potencial acumulado previamente. Este potencial compensa el costo real de recorrer y cortar múltiples nodos en el árbol.

7. ¿Qué errores de implementación detectaron los verificadores de invariantes?

Detectaron raíces que mantenían la marca `mark=True` tras ser promovidas en la extracción del mínimo. También encontraron referencias erróneas en los nodos padre y errores de grado tras realizar cortes simples.

8. ¿Recomendaría esta implementación para un sistema real en Python?

No en Python interpretado estándar, porque el manejo de punteros dinámicos anula su ventaja asintótica. Solo es útil si se implementa en lenguajes como C o C++ para procesar grafos muy densos.

5. Conclusiones

Implementamos el montículo de Fibonacci en Python y analizamos su comportamiento amortizado mediante la función de potencial $\Phi(H) = t(H) + 2m(H)$, contrastando su desempeño frente al módulo estándar `heapq` en operaciones básicas, disminuciones intensivas y el algoritmo de Dijkstra.

La batería de pruebas automáticas verificó el cumplimiento de las ocho invariantes estructurales: orden de montículo, consistencia en listas circulares, desmarcado estricto en raíces y grados disjuntos tras consolidar. Comprobamos que el costo amortizado $O(1)$ en inserción y disminución de clave permite acumular potencial suficiente para financiar las reestructuraciones profundas y cortes en cascada durante la extracción del mínimo en $O(\log n)$.

En el plano experimental, `heapq` fue sustancialmente más rápido en inserción y extracción ordinarias gracias a su implementación en lenguaje C sobre memoria contigua. Sin embargo, en el escenario de disminuciones intensivas ($q = 10n$), el montículo de Fibonacci superó a `heapq` en tiempo total (2 721 ms frente a 3 307 ms para 200 000 disminuciones), ya que `heapq` sufrió una degradación por acumulación de entradas obsoletas (alcanzando 148 731 entradas físicas frente a 10 000 activas). En Dijkstra ambas estructuras calcularon distancias mínimas idénticas en todas las instancias. Concluimos que el montículo de Fibonacci ofrece una ventaja clara en cargas dominadas por disminuciones de clave, aunque en entornos interpretados como Python la sobrecarga de punteros contrarresta parte de su beneficio asintótico frente a bibliotecas compiladas.